

Trabalho de Projecto - 1ª fase

1. Introdução

O projeto propõe a implementação de **ShoppingSO** – um sistema compacto para analisar compras de clientes em grande escala, construído de forma a demonstrar conceitos fundamentais de Sistemas Operativos. A ideia central é trabalhar com ficheiros “reais” de clientes (CSV com muitas linhas de compras) e mostrar, progressivamente, como dividir e coordenar trabalho entre processos e threads, como comunicar entre componentes, e como garantir consistência quando vários agentes acedem a recursos partilhados.

2. Objetivo

O principal objetivo deste projeto é desenhar e desenvolver um sistema híbrido, composto por um script em Bash e um programa em Python, que em conjunto permitam explorar e demonstrar os mecanismos de concorrência e paralelismo fornecidos pelos sistemas operativos. De forma separada, cada componente tem funções específicas:

- **Script Bash:** responsável por lançar os processos, cada um associado a um ficheiro de cliente (um CSV com milhares de registos de compras). O script assegura que apenas um número fixo de processos corre em simultâneo, ilustrando assim o controlo de concorrência no nível do sistema operativo.
- **Programa em Python:** cada processo executa o programa em Python, que por sua vez cria múltiplas threads para analisar os dados do ficheiro do cliente. Este paralelismo ao nível das threads divide a análise em subtarefas independentes, facilitando a exploração do modelo de partilha de memória e da necessidade (ou não) de sincronização.

No conjunto, o sistema demonstra a **coordenação entre dois níveis de concorrência**:

1. **Entre processos**, onde cada processo trata de um ficheiro de cliente distinto, garantindo escalabilidade com múltiplos ficheiros;
2. **Dentro de cada processo**, onde várias threads trabalham em paralelo sobre o mesmo ficheiro, cada uma com uma responsabilidade distinta:
 - **Thread 1:** identificar compras de artigos dispendiosos;
 - **Thread 2:** calcular o valor total das compras;
 - **Thread 3:** analisar o comportamento temporal das compras;

Cada ficheiro CSV contém informação essencial: **data da compra, nome do artigo e preço**, permitindo às *threads* realizar as suas análises.

3. Metodologia

O projeto será implementado em duas camadas principais: (1) o script Bash, responsável pela gestão de processos, e (2) o programa Python, responsável pela análise de cada ficheiro de cliente através de múltiplas threads.

3.1 Script Bash

O script em Bash é o ponto de entrada do sistema e executa as seguintes funções:

1. Percorre todos os ficheiros de clientes disponíveis numa pasta especificada.
2. Lança um processo Python independente para cada ficheiro, garantindo que, em cada instante, apenas um número máximo de processos é executado em paralelo (parâmetro configurável).
3. Aguarda a conclusão de um processo sempre que o limite de concorrência é atingido, recorrendo ao comando builtin `wait -n`.

3.2 Parâmetros do script Bash

O script deve ser invocado da seguinte forma:

```
./executar_analises.sh <pasta_dos_clientes> <max_processos>
```

- **pasta_dos_clientes**: caminho para a pasta onde se encontram os ficheiros CSV (um por cliente).
- **max_processos**: número máximo de processos a executar em simultâneo (ex.: 4).

O script garante que todos os ficheiros CSV da pasta são analisados. Nota: apenas podem usar comandos builtin de Bash.

4. Programa Python

Cada processo invoca o programa em Python, que recebe como argumento o ficheiro CSV do cliente a analisar. O programa divide o trabalho em **três threads** distintas, cada uma com uma função específica:

- **Thread 1 (artigos caros)**: identifica todas as compras caras, ie, cujo valor seja superior a 1000.0 €.
- **Thread 2 (total global)**: soma o valor total gasto pelo cliente em todas as compras.
- **Thread 3 (análise temporal)**: identifica todas as compras feitas nos dias especiais, ie, compras realizadas dias 29, 30 e 31 de qualquer mês.

As threads trabalham em paralelo, processando as linhas do ficheiro à medida que vão sendo disponibilizadas

pelo processo “main” via filas partilhadas. Assim, a main não deverá carregar todo o ficheiro em memória. Por razões pedagógicas, cada thread deverá “dormir” 10ms no fim de cada ciclo, e as filas deverão ter tamanho cinco.

4.1 Output

A thread 1 deverá imprimir a mensagem “Compra cara: produto -> valor€”, sempre que encontra um produto caro. A thread2 deverá imprimir a mensagem: “Total gasto: valor€”, depois de somar todas as compras. A thread3 deverá imprimir o texto: “Compra dia especial: produto -> data”, sempre que encontra um produto comprado num dia especial. As expressões `produto`, `valor` e `data` são *placeholders* que devem ser corretamente apresentados. Quando todas as threads terminarem, a main deverá imprimir a mensagem “Análise concluída”, e de forma similar, a main deverá imprimir “Análise iniciada.” quando é lançada.

Para se identificar o que cada processo/thread imprime, todas as mensagens impressas deverão ser prefixadas por “[N:D]”, onde N é o nome do ficheiro cliente em análise. D será T1, T2, T3 ou main, dependendo da thread que está a tentar imprimir.

4.2 Parâmetros do programa Python

O programa deve ser invocado da seguinte forma:

```
python3 analisar_cliente.py <caminho_ficheiro_csv>
```

caminho_ficheiro_csv: caminho para o ficheiro CSV correspondente a um cliente.

Exemplo de ficheiro CSV com a estrutura esperada:

```
Data,Hora,Produto,Preco
2026-04-23,23:36:06,PlacaGrafica-Amazon-VHD528,361.51
2026-03-27,04:05:20,Carregador-Intel-HWD278,421.43
2026-04-30,15:18:14,Rato-Xiaomi-KDF814,1395.13
...
```

Nota: apenas podem usar as bibliotecas referidas nas aulas TPs.

5. Integração

O ciclo completo de execução é o seguinte:

1. O utilizador invoca o script Bash, passando a pasta com ficheiros CSV e o limite de processos.
2. O script inicia até **N processos** em paralelo, cada um a executar o programa Python sobre um ficheiro de cliente.
3. Dentro de cada processo, o Python cria **3 threads**, cada uma responsável por uma dimensão da análise de compras.
4. O resultado é impresso no ecrã (na primeira fase do projeto) de forma independente para cada dimensão/cliente.

Exemplo de impressão no ecrã:

```
$ ./executar_analises.sh csvs 3
...
[customer1:main] Análise concluída.
[customer4:T2] Total gasto: 136894.46€
[customer4:main] Análise concluída.
[customer5:main] Análise iniciada.
[customer6:main] Análise iniciada.
[customer5:T3] Compra dia especial: Motherboard-Asus-A0Y883 -> 2026-04-29
[customer6:T1] Compra cara: Motherboard-Acer-BAC688 -> 1023.13€
...
```

6. Ficheiros Iniciais de Teste

Juntamente com o enunciado do projeto, serão disponibilizados um ficheiro ZIP (SO-TI-XX.zip) com a estrutura inicial do projeto e alguns ficheiros de texto que servirão para testarem a solução.

7. Entrega

A entrega do trabalho é realizada da seguinte forma:

- Os grupos inscrevem-se atempadamente, de acordo com as regras afixadas para o efeito, no Moodle.
- O grupo coloca os dois ficheiros do projeto – `analisar_cliente.py` e `executar_analises.sh` – numa diretoria cujo nome deve seguir exatamente o padrão SO-TI-XX (e.g., SO-TI-01 ou SO-TI-23).
- A diretoria será incluída num ficheiro ZIP cujo nome deve seguir exatamente o padrão grupo SO-TI-XX.zip, novamente onde XX corresponde ao número do vosso grupo. Este ficheiro deverá ser submetido no Moodle (apenas uma submissão por grupo).

De notar que o ficheiro ZIP entregue deve conter apenas a diretoria com os ficheiros, pois qualquer outro ficheiro será ignorado.

Não serão aceites trabalhos entregues por email nem por qualquer outro meio não definido nesta secção.

8. Avaliação dos Trabalhos

A avaliação do trabalho será realizada:

- Pelos alunos, através do preenchimento do formulário de contribuição de cada aluno no desenvolvimento do projeto. O formulário será disponibilizado no Moodle e preenchido após a entrega do projeto.
- Pelo corpo docente, sobre dois conjuntos de ficheiros de texto: os ficheiros de teste disponibilizados aos alunos e outros somente usados pelos docentes.

Para além dos testes a efetuar, os seguintes parâmetros serão avaliados: funcionalidade, estrutura, desempenho, algoritmia, comentários, clareza do código, validação dos parâmetros de entrada e tratamento de erros.

9. Plágios

Não é permitido aos alunos partilharem códigos com soluções, ainda que parciais, de nenhuma parte do projeto com outros alunos (nem através do Fórum da disciplina, nem por qualquer outro meio). Além disso, todos os códigos serão testados por um verificador de plágio. Caso alguma irregularidade seja encontrada, os projetos de todos os alunos envolvidos serão anulados e o caso será reportado aos órgãos responsáveis em Ciências@ULisboa.

Chamamos a atenção para o facto das plataformas generativas baseadas em Inteligência Artificial (e.g., o *ChatGPT* e o *GitHub Co-Pilot*) gerarem um número limitado de soluções diferentes para o mesmo problema, as quais podem ainda incluir padrões característicos deste tipo de ferramenta e que podem ser detetáveis pelos verificadores de plágio. Desta forma, recomendamos fortemente que os alunos não submetam trechos de código gerados por este tipo de ferramenta a fim de evitar riscos desnecessários.

Por fim, é responsabilidade de cada aluno garantir que a sua *home*, as suas diretorias e os seus ficheiros de código estão protegidos contra a leitura de outras pessoas (que não o utilizador dono dos mesmos). Por exemplo, se os ficheiros estiverem gravados na sua área de aluno nos servidores de Ciências@ULisboa, então todos os itens mencionados anteriormente devem ter as permissões de acesso **700**. Se os ficheiros estiverem no *GitHub*, garantam que o conteúdo do vosso repositório não esteja visível publicamente.